

Spring Framework Learning Topics for Real-Time Projects & Certification

1. CORE CONTAINER

1.1 IoC Container & Dependency Injection

- Understanding IoC (Inversion of Control) principle
- BeanFactory vs ApplicationContext
- Bean lifecycle and scopes (singleton, prototype, request, session, application)
- Constructor-based dependency injection
- Setter-based dependency injection
- Field injection and best practices
- Circular dependencies and resolution
- @Autowired, @Inject, @Resource annotations
- @Qualifier and @Primary for disambiguation
- Optional dependencies with @Autowired(required=false)

1.2 Bean Configuration

- Java-based configuration (@Configuration, @Bean)
- Annotation-based configuration (@Component, @Service, @Repository, @Controller)
- XML-based configuration (legacy but important)
- Component scanning and filters
- @ComponentScan with basePackages and basePackageClasses
- Bean naming strategies
- Lazy initialization (@Lazy)
- Bean definition inheritance
- Factory beans and FactoryBean interface

1.3 Advanced Container Features

- ApplicationContext events (ContextRefreshedEvent, ContextStartedEvent, etc.)
- Custom events with ApplicationEventPublisher
- @EventListener and event handling
- Environment abstraction and profiles
- @Profile for environment-specific beans
- Property sources and @PropertySource
- @Value annotation and SpEL (Spring Expression Language)
- Type conversion and Converter SPI
- Validation with JSR-303/JSR-380 (Bean Validation)

1.4 AOP (Aspect-Oriented Programming)

- AOP concepts: Aspect, Join Point, Advice, Pointcut, Weaving
- @Aspect annotation

- Advice types: @Before, @After, @AfterReturning, @AfterThrowing, @Around
- Pointcut expressions and designators
- Introduction and @DeclareParents
- Proxy mechanisms (JDK dynamic proxy vs CGLIB)
- @EnableAspectJAutoProxy
- AOP use cases: logging, security, transactions, caching

1.5 Resources & Data Binding

- Resource abstraction (UrlResource, ClassPathResource, FileSystemResource)
- ResourceLoader and ResourcePatternResolver
- Data binding with DataBinder
- PropertyEditor and custom editors
- ConversionService and Formatter
- BeanWrapper for property manipulation

1.6 Spring Core Utilities

- StringUtils, CollectionUtils, ObjectUtils
- ClassUtils and ReflectionUtils
- FileCopyUtils and StreamUtils
- Assert class for preconditions

2. TESTING

2.1 Unit Testing

- Testing with JUnit 5 (Jupiter)
- Mockito integration
- @Mock, @InjectMocks, @Spy annotations
- Testing beans in isolation
- Constructor injection for testability
- Test doubles: mocks, stubs, fakes

2.2 Integration Testing

- Spring TestContext Framework
- @SpringBootTest for full application context
- @WebMvcTest for controller layer testing
- @DataJpaTest for repository layer testing
- @RestClientTest for REST client testing
- @JsonTest for JSON serialization testing
- TestPropertySource and @TestPropertySource
- @DirtiesContext for context management

2.3 Test Configuration

- @TestConfiguration for test-specific beans
- @MockBean and @SpyBean in Spring Boot

- Profile-specific test configurations
- `@ActiveProfiles` for test profiles
- Embedded databases for testing (H2, HSQLDB)
- Test fixtures and `@BeforeEach`, `@AfterEach`

2.4 Web Layer Testing

- `MockMvc` for testing MVC controllers
- `@AutoConfigureMockMvc`
- Testing REST endpoints with `MockMvc`
- Testing request/response with `MockHttpServletRequest/Response`
- `WebTestClient` for reactive applications
- `RestTemplate` and `TestRestTemplate`
- `@LocalServerPort` for random port testing

2.5 Advanced Testing

- `Testcontainers` for integration testing
- Transaction management in tests (`@Transactional`, `@Rollback`)
- Testing async methods with `@Async`
- Testing scheduled tasks
- Testing security with `@WithMockUser`, `@WithUserDetails`
- Performance testing and benchmarking
- Test slices and selective context loading

3. DATA ACCESS

3.1 JDBC & DataSource

- `JdbcTemplate` and `NamedParameterJdbcTemplate`
- `RowMapper`, `ResultSetExtractor`, `RowCallbackHandler`
- `DataSource` configuration
- Connection pooling (HikariCP, Tomcat JDBC)
- Batch operations with `JdbcTemplate`
- `SimpleJdbcInsert` and `SimpleJdbcCall`
- Embedded database support

3.2 Transaction Management

- `@Transactional` annotation
- Transaction propagation levels (REQUIRED, REQUIRES_NEW, NESTED, etc.)
- Transaction isolation levels
- Rollback rules and exceptions
- Programmatic transaction management (`TransactionTemplate`)
- `PlatformTransactionManager`
- `@EnableTransactionManagement`
- Read-only transactions and optimization
- Transaction synchronization and callbacks

3.3 Spring Data JPA

- @Entity, @Table, @Column annotations
- Primary keys: @Id, @GeneratedValue
- Relationships: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany
- Cascade types and fetch strategies (LAZY, EAGER)
- JpaRepository and CrudRepository
- Query methods and naming conventions
- @Query with JPQL and native SQL
- @Modifying for update/delete queries
- Pagination and sorting (Pageable, Sort)
- Specifications for dynamic queries
- Projections and DTOs
- Auditing with @CreatedDate, @LastModifiedDate, @CreatedBy, @LastModifiedBy
- @EntityGraph for fetch optimization

3.4 Spring Data Commons

- Repository abstraction
- Custom repository implementations
- Query by Example (QBE)
- QueryDSL integration
- Repository events and @RepositoryEventHandler
- Web support: Pageable and Sort as controller parameters
- Domain class converter

3.5 Spring Data REST

- Exposing repositories as REST endpoints
- HATEOAS support
- Customizing endpoints with @RepositoryRestResource
- Projections and excerpts
- Event handlers for REST operations
- Validators and custom controllers

3.6 Caching

- @EnableCaching
- @Cacheable, @CachePut, @CacheEvict
- Cache managers (ConcurrentMapCacheManager, EhCache, Redis)
- Cache key generation and SpEL
- Conditional caching
- Cache synchronization

3.7 Other Data Access Technologies

- Spring Data MongoDB
- Spring Data Redis
- Spring Data Elasticsearch

- R2DBC for reactive relational databases
- NoSQL database integration

4. WEB SERVLET (Spring MVC)

4.1 MVC Fundamentals

- DispatcherServlet and request processing flow
- @Controller and @RestController
- @RequestMapping and HTTP method variants (@GetMapping, @PostMapping, etc.)
- Handler methods and method arguments
- @PathVariable for URI templates
- @RequestParam for query parameters
- @RequestBody for request payload
- @ResponseBody for response payload
- Model, ModelMap, and ModelAndView
- RedirectAttributes for flash attributes

4.2 Request Handling

- Content negotiation and produces/consumes
- @RequestHeader and @CookieValue
- HttpServletRequest and HttpServletResponse
- @SessionAttribute and @SessionAttributes
- @ModelAttribute for form binding
- Validation with @Valid and BindingResult
- Type conversion and formatting
- Matrix variables with @MatrixVariable
- Multipart file upload with MultipartFile

4.3 Response Handling

- ResponseEntity for full HTTP response control
- HttpStatus codes
- @ResponseStatus annotation
- HttpHeaders manipulation
- Streaming responses
- Server-Sent Events (SSE)
- Content negotiation (JSON, XML, etc.)

4.4 REST API Development

- RESTful principles and best practices
- Resource naming conventions
- HATEOAS with Spring HATEOAS
- EntityModel, CollectionModel, RepresentationModel
- Hypermedia links and Link class
- API versioning strategies

- Richardson Maturity Model

4.5 Exception Handling

- @ExceptionHandler at controller level
- @ControllerAdvice for global exception handling
- @RestControllerAdvice
- ResponseEntityExceptionHandler
- Custom error responses
- Problem Details (RFC 7807) support
- Error attributes customization

4.6 Validation

- JSR-303/JSR-380 Bean Validation
- @NotNull, @NotEmpty, @NotBlank, @Size, @Min, @Max, @Email, @Pattern
- Custom validators with ConstraintValidator
- Validation groups
- @Validated for method-level validation
- Programmatic validation with Validator

4.7 Interceptors & Filters

- HandlerInterceptor interface
- preHandle, postHandle, afterCompletion methods
- WebMvcConfigurer for interceptor registration
- Filter vs Interceptor differences
- CORS configuration with @CrossOrigin
- Global CORS configuration

4.8 View Technologies

- Thymeleaf integration
- JSP and JSTL (legacy)
- FreeMarker and Velocity
- View resolvers
- Static resources handling
- Internationalization (i18n) with MessageSource
- Locale resolution strategies

4.9 Web Configuration

- WebMvcConfigurer interface
- @EnableWebMvc
- Custom argument resolvers
- Custom message converters (Jackson, Gson, XML)
- Path matching configuration
- Static resource handlers
- View controllers

- Async request processing with DeferredResult and Callable

4.10 File Upload/Download

- MultipartResolver configuration
- File upload with @RequestParam MultipartFile
- Multiple file uploads
- File download with ResponseEntity
- Streaming large files
- File validation and size limits

4.11 WebSocket Support

- @EnableWebSocket and @EnableWebSocketMessageBroker
- WebSocket handlers
- STOMP protocol support
- Message mapping with @MessageMapping
- @SendTo and @SendToUser
- SimpMessagingTemplate for programmatic messaging

5. WEB REACTIVE (Spring WebFlux)

5.1 Reactive Programming Fundamentals

- Reactive Streams specification
- Project Reactor: Mono and Flux
- Operators: map, flatMap, filter, zip, merge, concat
- Backpressure handling
- Hot vs Cold publishers
- Schedulers and threading model
- Error handling: onErrorReturn, onErrorResume, onErrorMap
- Retry and timeout strategies

5.2 WebFlux Core

- Reactive vs Servlet stack differences
- Annotated controllers with @RestController
- Functional endpoints with RouterFunction
- HandlerFunction and ServerRequest/ServerResponse
- WebClient for reactive HTTP clients
- Reactive request/response handling
- @RequestBody with Mono/Flux
- @ResponseBody with Mono/Flux

5.3 Reactive Data Access

- Spring Data R2DBC
- ReactiveCrudRepository and ReactiveSortingRepository
- Reactive transactions with @Transactional

- DatabaseClient for custom queries
- Reactive MongoDB with ReactiveMongoRepository
- Reactive Redis operations
- Reactive Cassandra support

5.4 Functional Endpoints

- RouterFunction and route definitions
- RequestPredicate and request matching
- Nested routes and route composition
- Filter functions
- HandlerFilterFunction
- Functional vs annotated controllers comparison

5.5 WebClient

- Creating WebClient instances
- GET, POST, PUT, DELETE requests
- Request headers and body
- Response handling with bodyToMono/bodyToFlux
- Error handling and status code checking
- Exchange strategies and codecs
- Client filters with ExchangeFilterFunction
- Retry and timeout configuration

5.6 Reactive Security

- Spring Security with WebFlux
- SecurityWebFilterChain
- Reactive authentication and authorization
- @EnableWebFluxSecurity
- ReactiveAuthenticationManager
- ReactiveUserDetailsService
- JWT authentication in reactive apps
- Method security with @PreAuthorize, @PostAuthorize

5.7 Server-Sent Events (SSE)

- Streaming with Flux and MediaType.TEXT_EVENT_STREAM
- SSE endpoint creation
- Client consumption of SSE
- Heartbeat and connection management

5.8 WebSocket in WebFlux

- Reactive WebSocket support
- WebSocketHandler interface
- WebSocket session management
- Message handling with Flux

5.9 Testing Reactive Applications

- WebClient for integration testing
- StepVerifier for testing Mono/Flux
- Testing reactive repositories
- Virtual time with StepVerifier.withVirtualTime
- Testing backpressure scenarios
- Mocking reactive components

5.10 Performance & Best Practices

- Non-blocking I/O benefits
- When to use reactive vs servlet stack
- Avoiding blocking calls in reactive chain
- Thread pool configuration
- Memory management with reactive streams
- Debugging reactive applications
- Reactive metrics and monitoring

6. SPRING BOOT ESSENTIALS

6.1 Spring Boot Fundamentals

- Auto-configuration mechanism
- @SpringBootApplication annotation
- Starter dependencies
- application.properties and application.yml
- Configuration properties with @ConfigurationProperties
- Profile-specific configurations
- Externalized configuration
- Command-line arguments and environment variables

6.2 Spring Boot Actuator

- Enabling Actuator with spring-boot-starter-actuator
- Health indicators and custom health checks
- Metrics with Micrometer
- Info endpoint customization
- Management endpoints (/actuator/*)
- Securing actuator endpoints
- Custom endpoints with @Endpoint
- Prometheus integration for monitoring

6.3 Spring Boot DevTools

- Automatic restart
- LiveReload support
- Property defaults for development
- Remote debugging

6.4 Packaging & Deployment

- Executable JAR and WAR packaging
- Maven and Gradle plugins
- Docker containerization
- Cloud deployment (AWS, Azure, GCP)
- Kubernetes deployment
- Environment-specific builds

7. SECURITY

7.1 Spring Security Basics

- Authentication vs Authorization
- SecurityContext and SecurityContextHolder
- @EnableWebSecurity and @EnableGlobalMethodSecurity
- UserDetailsService and UserDetails
- PasswordEncoder (BCrypt, Argon2, etc.)
- In-memory, JDBC, and custom authentication

7.2 Web Security

- HttpSecurity configuration
- URL-based authorization with antMatchers
- CSRF protection
- Session management strategies
- Remember-me authentication
- Logout handling
- Security filters chain

7.3 Method Security

- @PreAuthorize and @PostAuthorize
- @Secured and @RolesAllowed
- SpEL expressions in security annotations
- Method-level security with AOP

7.4 OAuth2 & JWT

- OAuth2 authorization flows
- Resource server configuration
- Authorization server setup
- JWT token generation and validation
- @EnableResourceServer and @EnableAuthorizationServer
- Spring Security OAuth2 Client
- Social login integration

8. MICROSERVICES & CLOUD

8.1 Spring Cloud Basics

- Service discovery with Eureka
- Client-side load balancing with Ribbon/LoadBalancer
- API Gateway with Spring Cloud Gateway
- Circuit breaker with Resilience4j
- Distributed configuration with Spring Cloud Config
- Distributed tracing with Sleuth and Zipkin

8.2 Messaging

- Spring AMQP with RabbitMQ
- Spring Kafka integration
- @KafkaListener and KafkaTemplate
- Message-driven POJOs
- Event-driven architecture

9. BEST PRACTICES & PATTERNS

- Dependency injection best practices
- Constructor injection over field injection
- Immutable configuration properties
- Exception handling strategies
- Logging with SLF4J and Logback
- API documentation with OpenAPI/Swagger
- DTO pattern and MapStruct
- Repository pattern
- Service layer design
- Clean architecture principles
- SOLID principles in Spring applications

10. CERTIFICATION PREPARATION TIPS

1. **Hands-on Practice:** Build real projects covering all modules
2. **Official Documentation:** Read Spring Framework and Spring Boot reference docs
3. **Mock Tests:** Practice with sample certification questions
4. **Code Reviews:** Review open-source Spring projects on GitHub
5. **Focus Areas:** IoC/DI, AOP, Data Access, MVC, Security, Testing
6. **Spring Boot:** Understand auto-configuration and starters deeply
7. **Troubleshooting:** Practice debugging common Spring issues
8. **Performance:** Understand caching, connection pooling, lazy loading
9. **Security:** Master authentication, authorization, and OAuth2
10. **Reactive:** Understand when and how to use reactive programming

Recommended Learning Path

1. **Week 1-2:** Core Container (IoC, DI, Bean lifecycle, Configuration)
2. **Week 3:** AOP and advanced container features

3. **Week 4:** Testing (Unit and Integration)
4. **Week 5-6:** Data Access (JDBC, JPA, Transactions)
5. **Week 7-8:** Web Servlet (Spring MVC, REST APIs)
6. **Week 9:** Security fundamentals
7. **Week 10:** Web Reactive (WebFlux, Reactor)
8. **Week 11:** Spring Boot deep dive
9. **Week 12:** Microservices and Cloud patterns
10. **Week 13-14:** Practice projects and mock tests

Resources

- Official Spring Framework Documentation: <https://docs.spring.io/spring-framework/reference/>
 - Spring Boot Documentation: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
 - Spring Academy: <https://spring.academy/>
 - Baeldung Spring Tutorials: <https://www.baeldung.com/spring-tutorial>
 - Spring Certification Study Guide (if available)
 - GitHub Spring Projects: <https://github.com/spring-projects>
-

Note: This comprehensive list covers topics essential for both real-time project development and Spring Professional Certification. Focus on understanding concepts deeply with hands-on practice rather than just memorizing theory.